

Державний вищий навчальний заклад
Ужгородський національний університет
Факультет інформаційних технологій
Кафедра програмного забезпечення систем

ЛАБОРАТОРНА РОБОТА № 4.1

Тема: Апроксимація функцій методами інтерполяції та МНК

Виконав:
студент II курсу
спеціальності: 121 інженерія
програмного забезпечення
Романюк Артем Романович

Ужгород-2026

Мета: засвоїти підходи до апроксимації даних, зокрема інтерполяційний поліном Лагранжа та метод найменших квадратів, а також набути практичних навичок їх реалізації мовою програмування JavaScript і візуалізації отриманих результатів.

Завдання до роботи:

- Розробка технічного завдання.
- Вибір та опис методології розробки ПЗ. При виборі методології обрати методологію відмінну від попередніх лабораторних робіт.

Хід роботи

1. Розробка технічного завдання

Технічне завдання

Зміст:

1. Загальна інформація
2. Мета проекту
3. Основні функціональні вимоги

1. Загальна інформація

Назва проекту: Програмний модуль для апроксимації функцій методами інтерполяції та МНК.

Галузь застосування: Навчальний процес дисципліни «Методи програмної інженерії для розв'язування прикладних задач».

Цільова платформа: Веб-середовище (браузер).

Інструментарій розробки:

- Мова програмування: JavaScript.
- Розмітка та стилі: HTML5, CSS3.
- Візуалізація: бібліотека D3.js.

2. Мета проекту

Метою розробки є створення програмного продукту, який дозволяє:

- Реалізувати побудову інтерполяційного полінома Лагранжа та у формі Ньютона за заданим табличним набором точок $\{(x_i, y_i)\}$.
- Виконати апроксимацію даних методом найменших квадратів (МНК) із вибором ступеня полінома $m=2, 3, 4$.
- Візуалізувати результати: відобразити вихідні точки, криву інтерполяції та криву МНК, а також діаграму залишків r_i .
- Забезпечити анімацію покрокового додавання вузлів інтерполяції та «виростання» кривої МНК.

3. Основні функціональні вимоги

Організація інтерфейсу: адаптивне полотно D3.js, перемикач «Інтерполяція / МНК / Усі», осі з підписами й сіткою, легенда, підказки при наведенні (координати, значення похибки).

Введення даних: можливість введення набору точок $\{(x_i, y_i)\}$ через форму або вибір варіанта зі списку.

Алгоритмічне забезпечення:

- Інтерполяція Лагранжа — точне відтворення значень функції в усіх заданих вузлах.
- Інтерполяція Ньютона — аналогічна точна інтерполяція із використанням таблиці поділених різниць.
- МНК-апроксимація — побудова дизайн-матриці, обчислення коефіцієнтів стійким методом (без нормальних рівнянь у підсумку), обчислення залишків $r_i = y_i - \hat{y}_i$.

Обробка та вивід результатів:

- Відображення графіка функції з точками, кривою інтерполяції, кривою МНК.
- Побудова діаграми залишків (вертикальні відрізки від точки до кривої МНК).
- Обчислення та відображення метрик якості: RMSE, R^2 .

Анімація:

- Інтерполяція: покрокове додавання вузлів із плавним оновленням кривої та виділенням активного вузла.
- МНК: плавне «виростання» полінома від горизонтальної лінії до фінальної кривої, поступова поява залишків.
- Вимоги до коду: Git-коміт для кожного логічного етапу з описовим повідомленням.

2. Вибір та опис методології розробки ПЗ. Розробка методики вирішення задачі

Для реалізації проекту обрано каскадну методологію Waterfall. На відміну від ітеративних підходів (Scrum, Kanban), Waterfall передбачає чітко визначені послідовні фази, кожна з яких повністю завершується перед переходом до наступної. Для даного проекту це є доцільним, оскільки вимоги до алгоритмів інтерполяції та МНК є чітко сформульованими у технічному завданні та не змінюватимуться в процесі розробки.

Фази Waterfall для даного проекту:

Фаза	Зміст робіт	Результат
1. Аналіз вимог	Опис вхідних даних $\{(x_i, y_i)\}$, визначення методів (Лагранж, Ньютон, МНК), формулювання функціональних і нефункціональних вимог	Затверджене ТЗ
2. Проектування	Проектування архітектури модулів: введення даних, обчислення, візуалізація. Розробка діаграми взаємодії компонентів	Діаграма компонентів, структура файлів
3. Реалізація	Розробка модулів у послідовності: введення даних $\rightarrow$ інтерполяція Лагранжа $\rightarrow$ інтерполяція	Програмний код JS, HTML, CSS

	Ньютона → МНК → візуалізація D3.js → анімація	
4. Тестування	Перевірка коректності алгоритмів на тестових наборах, порівняння результатів із аналітичними значеннями, перевірка метрик RMSE/R ²	Протокол тестування
5. Розгортання	Завантаження на GitHub, оформлення звіту, фінальний коміт	Репозиторій, звіт

3. Методика вирішення задачі апроксимації/інтерполяції

Метод 1: Інтерполяція Лагранжа

За $n+1$ вузловими точками (x_i, y_i) будується поліном ступеня n , що точно проходить через всі задані точки:

$$P(x) = \sum y_k \cdot L_k(x), \text{ де } L_k(x) = \prod (x - x_j) / (x_k - x_j), \quad j \neq k$$

Перевага: проста реалізація. Недолік: при великій кількості вузлів виникає ефект Рунге (осциляція на краях).

Метод 2: Інтерполяція Ньютона (поділені різниці)

Поліном записується у формі:

$$P(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \dots + a_n \prod (x-x_j)$$

де коефіцієнти $a_k = f[x_0, x_1, \dots, x_k]$ — поділені різниці. Форма Ньютона зручна тим, що при додаванні нового вузла не потрібно перераховувати всі коефіцієнти.

Таблиця поділених різниць (демонстраційний набір: $x_i = \{0,1,2,3,4\}$, $y_i = \{1,3,2,5,4\}$):

Вузол	$f[x_i]$	$f[\cdot, \cdot]$	$f[\cdot, \cdot, \cdot]$	$f[\cdot, \cdot, \cdot, \cdot]$	$f[\cdot, \cdot, \cdot, \cdot, \cdot]$
$x_0=0$	$f[x_0]=1$	$f[x_0, x_1]=2$	$f[x_0, x_1, x_2]=-1.5$	$f[x_0..x_3]=1.333$	$f[x_0..x_4]=-0.583$
$x_1=1$	$f[x_1]=3$	$f[x_1, x_2]=-1$	$f[x_1, x_2, x_3]=2$	$f[x_1..x_4]=-1.5$	

$x_2=2$	$f[x_2]=2$	$f[x_2,x_3]=3$	$f[x_2,x_3,x_4]=-0.5$		
$x_3=3$	$f[x_3]=5$	$f[x_3,x_4]=-1$			
$x_4=4$	$f[x_4]=4$				

Коефіцієнти полінома Ньютона: $a_0=1, a_1=2, a_2=-1.5, a_3=1.333, a_4=-0.583$

Метод 3: Апроксимація МНК (поліноміальна регресія)

На відміну від інтерполяції, МНК не вимагає точного проходження через вузли, а мінімізує суму квадратів відхилень:

$$\Delta = \sum_i (y_i - g(x_i; \theta))^2 \rightarrow \min$$

Алгоритм:

Побудувати дизайн-матрицю Φ розміру $(n \times (m+1))$, де m — обраний ступінь полінома.

Знайти вектор коефіцієнтів $\theta = (\Phi^T \Phi)^{-1} \Phi^T y$ (через QR-розклад для числової стійкості).

Обчислити залишки $r_i = y_i - \hat{y}_i$ та метрики $RMSE = \sqrt{(\sum r_i^2 / n)}$, $R^2 = 1 - \sum r_i^2 / \sum (y_i - \bar{y})^2$.

Вибір ступеня m : при малому m — недоапроксимація, при $m \geq n-1$ — вироджується в інтерполяцію та можливе перенавчання.

4. Розробка діаграми взаємодії компонентів системи

Діаграма відображає послідовність взаємодії між компонентами при виконанні інтерполяції та МНК:

Користувач		UI (HTML/CSS)		Controller (JS)		D3.js Renderer
Вводить точки $\{(x_i, y_i)\}$	→	Форма Select варіанта	/			

		Валідація вхідних даних	→	runInterpolati on() / runLSQ()		
				Обчислення коефіцієнтів, залишків, RMSE, R ²	→	drawChart(poi nts, curve, residuals)
						Анімація кривої + залишки на SVG
Бачить графік, метрики, анімацію	←	Оновлення UI (перемикач, легенда, tooltip)	←	Повертає результати	←	

Висновки

Під час виконання лабораторної роботи №4.1 я опанував методи апроксимації функцій за табличними даними та набув практичних навичок їх реалізації на JavaScript. У ході роботи було досягнуто наступних результатів:

Проектування системи: розроблено технічне завдання для інтерактивного веб-додатка, що реалізує два підходи до наближення функцій - інтерполяцію та апроксимацію МНК. Вибір методології Waterfall дозволив структурувати процес розробки на п'ять чітких послідовних фаз, що забезпечує повну документованість кожного етапу та відповідність результату початковим вимогам.

Аналіз методів: проведено порівняльний аналіз методів інтерполяції Лагранжа та Ньютона. Встановлено, що форма Ньютона є ефективнішою при поступовому додаванні вузлів, оскільки не потребує повного перерахунку

коефіцієнтів. Обидва методи гарантують точне проходження через задані вузли, проте при великій їх кількості виникає ефект Рунге.

Практичне застосування МНК: побудовано алгоритм апроксимації з використанням дизайн-матриці та QR-розкладу для числової стійкості. Показано, що вибір ступеня полінома m суттєво впливає на якість наближення: при $m < n-1$ виникає недоапроксимація, при $m = n-1$ метод вироджується в інтерполяцію.

Візуалізація та логіка: розроблена діаграма взаємодії компонентів чітко відображає архітектуру системи: UI приймає дані, контролер виконує обчислення, D3.js Renderer будує графіки з анімацією та залишками. Перемикач «Інтерполяція / МНК / Усі» забезпечує наочне порівняння двох підходів.

Отримані знання з інтерполяції, апроксимації та візуалізації є фундаментальними для задач моделювання, комп'ютерної графіки та чисельних методів у подальшій розробці програмного забезпечення.